

miniLLM Learning Notes

These notes show my full learning path for miniLLM, an autoregressive language-model training workflow written in C11.

1. What Configuration Values Does miniLLM Use?

There are a few different core structs.

MiniLLMConfig

`include/minillm.h`

```
typedef struct {
    int vocab_size;          /* The byte tokenizer is fixed at 256 tokens. */
    int context_length;     /* Maximum tokens visible at once. */
    int d_model;            /* Vector dimension of each token representation. */
    int n_heads;            /* Attention heads; d_model must be divisible by n_heads. */
    int n_layers;           /* Number of Transformer blocks. */
    int hidden_dim;         /* MLP intermediate width, usually 2-4 times d_model. */
    uint64_t seed;         /* Seed for initialization and sampling. */
} MiniLLMConfig;
```

- **vocab_size**: The number of distinct token IDs. Because one byte is used as one token, there are $2^8 = 256$ possible byte values.
- **context_length**: The maximum number of token positions processed in one sequence during a forward/**backward pass**—the **context window**.
- For each training sample, we select **context_length** + 1 tokens. The first **context_length** tokens are the inputs, and the same sequence shifted one token to the right provides the targets.
- **d_model**: The number of dimensions in each token representation. For example, a token can be embedded as a one-dimensional array of 32 numbers. After each **Transformer block**, the hidden state is still D-dimensional but also contains contextual information.
- **n_heads**: The number of **attention heads**. **d_model** must be divisible by **n_heads**, so each head normally uses **d_model / n_heads** dimensions.
- **n_layers**: The number of **Transformer blocks**.
- **hidden_dim**: The width of the **MLP**'s intermediate layer, usually 2–4 times **d_model**.
- **seed**: The random **seed** used for parameter initialization and sampling.

MiniLLMOptimizer

```
typedef struct {
    float learning_rate;
    float beta1;
    float beta2;
    float epsilon;
    float weight_decay;
    float grad_clip; /* Gradient-norm limit; <= 0 disables clipping. */
} MiniLLMOptimizer;
```

These are the **optimizer hyperparameters**.

Tensor

`src/internal/minillm_internal.h`

```
typedef struct {
    size_t count;
    float *value; /* Parameter values, w */
    float *grad; /* Gradients, dw */
    float *m; /* Adam first-moment estimates */
    float *v; /* Adam second-moment estimates */
} Tensor;
```

- **count**: The total number of scalar values in the **tensor**.

- **value**, **grad**, **m**, and **v** each point to a separate contiguous one-dimensional array containing **count** floats. A matrix or higher-dimensional shape is imposed by the code that uses the **tensor**; these are not pointers to two-dimensional arrays.

Transformer Block Parameters

```
typedef struct {
    Tensor rms1_gain; /* [D] */
    Tensor wq;         /* [D, D] */
    Tensor wk;         /* [D, D] */
    Tensor wv;         /* [D, D] */
    Tensor wo;         /* [D, D] */
    Tensor rms2_gain; /* [D] */
    Tensor w1;         /* [D, F] */
    Tensor b1;         /* [F] */
    Tensor w2;         /* [F, D] */
    Tensor b2;         /* [D] */
} Block;
```

- **D**: The model's hidden dimension, for example 128.
- **F**: The **MLP**'s intermediate dimension, for example 512.
- **T**: The context length.

Roles of the block parameters

- **rms1_gain**: **Normalization gain** before **attention**.
- **wq / wk / wv**: Produce the **Query**, **Key**, and **Value** projections.
- **wo**: Mixes and projects the **attention** result.
- **rms2_gain**: **Normalization gain** before the **MLP**.
- **w1 / b1**: Expands from D to the **MLP** intermediate dimension F.
- **w2 / b2**: Projects from F back to the model dimension D.

2. Linear Layers and the MLP

In this project, the **MLP** is built from two **linear layers** with a nonlinear activation between them.

Forward Pass

```
void minillm_linear_forward(const float *x, const Tensor *weight,
                           const Tensor *bias, float *y,
                           int rows, int input_dim, int output_dim) {
    int row;
    for (row = 0; row < rows; ++row) {
        int out;
        for (out = 0; out < output_dim; ++out) {
            int in;
            float sum = bias != NULL ? bias->value[out] : 0.0f;
            for (in = 0; in < input_dim; ++in) {
                sum += x[row * input_dim + in] *
                    weight->value[in * output_dim + out];
            }
            y[row * output_dim + out] = sum;
        }
    }
}
```

Shape Convention

In this project, the **linear layer** uses the following shapes:

- **Input x**: [rows, input_dim]
- **Weight W**: [input_dim, output_dim]
- **Bias b**: [output_dim] (optional)
- **Output y**: [rows, output_dim]

In other words:

```
y[row][out] = x[row][0] * W[0][out]
              + x[row][1] * W[1][out]
              + ...
              + bias[out]
```

$(m \times n)(n \times p) \rightarrow (m \times p)$

A way I remember it

The two inner dimensions must match, and the two outer dimensions determine the output shape.

Example: One Row (One Token)

Suppose the input is:

```
x = [2, 3]
```

We want an output of length 2:

```
y = [?, ?]
```

The **weight matrix** and **bias** are:

```
W = [[1, 4],
     [2, 5]]
```

```
b = [10, 20]
```

The linear **forward pass** computes $y[0]$ and $y[1]$ separately.

First output

```
y[0] = x[0] * W[0][0] + x[1] * W[1][0] + b[0]
      = 2 * 1 + 3 * 2 + 10
```

= 18

Second output

```
y[1] = x[0] * W[0][1] + x[1] * W[1][1] + b[1]
      = 2 * 4 + 3 * 5 + 20
      = 43
```

Therefore:

```
y = [18, 43]
```

Why Is There an Outer row Loop?

Why does the code have an outer row loop?

Because it normally processes multiple token rows at once. Every token row uses the same **linear-layer** weights.

Projection to the Vocabulary

For the final projection to the vocabulary:

```
final_norm   : [T, D]
output_weight : [D, 256]
output_bias   : [256]
logits       : [T, 256]
```

Here, 256 is the vocabulary size. Each row is one token's D-dimensional feature vector. A [1, D] row multiplied by [D, 256] produces [1, 256], and processing T rows produces [T, 256].

These 256 values are **logits**—unnormalized scores. Applying **softmax** converts them into a **probability distribution** over the 256 byte tokens.

Backward Pass

```
void minillm_linear_backward(const float *x, Tensor *weight, Tensor *bias,
                             const float *dy, float *dx,
                             int rows, int input_dim, int output_dim) {
    int row;
    for (row = 0; row < rows; ++row) {
        int out;
        for (out = 0; out < output_dim; ++out) {
            int in;
            const float gradient = dy[row * output_dim + out];
            if (bias != NULL) {
                bias->grad[out] += gradient;
            }
            for (in = 0; in < input_dim; ++in) {
                weight->grad[in * output_dim + out] +=
                    x[row * input_dim + in] * gradient;
                dx[row * input_dim + in] +=
                    weight->value[in * output_dim + out] * gradient;
            }
        }
    }
}
```

dy has exactly the same shape as the forward output y.

After the **forward pass**, the **backward pass** uses the **upstream gradient** dy to compute three things:

- **dx**: The **gradient** with respect to the input x. It is passed to the preceding layer as that layer's **upstream gradient**.
- **weight->grad**: The **gradient** with respect to the **weight matrix** W.
- **bias->grad**: The **gradient** with respect to the **bias** b.

The forward equation is:

$$y = xW + b$$

For one token row and one output position:

$$y[\text{row}][\text{out}] = \sum x[\text{row}][\text{in}] * W[\text{in}][\text{out}] + b[\text{out}]$$

The backward equations are:

$$dW[\text{in}][\text{out}] += x[\text{row}][\text{in}] * dy[\text{row}][\text{out}]$$

$$db[\text{out}] += dy[\text{row}][\text{out}]$$

$$dx[\text{row}][\text{in}] += W[\text{in}][\text{out}] * dy[\text{row}][\text{out}]$$

The += operators matter because gradients accumulate across token rows and output dimensions.

What Does **gradient** Mean?

const float **gradient** = dy[row * output_dim + out];

dy is the **gradient** passed back from the later part of the network:

$$dy[\text{row}][\text{out}] = \partial \text{loss} / \partial y[\text{row}][\text{out}]$$

In other words, it measures how a small change in the current output $y[\text{row}][\text{out}]$ would change the final loss.

Next: use the **optimizer** to update the weights and biases.

3, RMSNorm

Force the signal to stay in a healthy **scale** rh is $-1 \sim 1$

```
void minillm_rms_forward(const float *x, const float *gain, float *y,
                        float *inverse_rms, int rows, int columns) {
    int row;
    for (row = 0; row < rows; ++row) {
        int col;
        float square_sum = 0.0f;
        for (col = 0; col < columns; ++col) {
            const float value = x[row * columns + col];
            square_sum += value * value;
        }
        inverse_rms[row] = 1.0f /
            sqrtf(square_sum / (float)columns + MINILLM_RMS_EPSILON);
        for (col = 0; col < columns; ++col) {
            y[row * columns + col] = x[row * columns + col] *
                inverse_rms[row] * gain[col];
        }
    }
}
```

X: input with shape [t,d]

Rows: amount of tokens t

Columns: hidden dimensions d

So the shape of the input is [t,d]

Gain: a 1d array with length d-> will be shared for all the tokens

The only learnable factor in norm layer

它的作用：**归一化**会把向量强行拉到**均方根**为 1。但这可能破坏了模型表达能力，所以模型需要一个参数来把数值再“缩放”到它觉得合适的范围内。

逐元素乘法 token 每一个维度对应一个

float ***inverse_rms**:

To save a middle result for calculation in **backward pass**

Only 3 steps:

For every row (token), square each of them and sum them up

$$\text{inverse_rms} = \frac{1}{\sqrt{\frac{x_0^2 + x_1^2 + \dots + x_{31}^2}{32} + \epsilon}}$$

And then squareroot that value and do

And then we save this

And then 每一个 token 里面的每一个维度*这个 token 的 inverserm 再***gain** 里面的对应的那一个数值

```
void minillm_rms_backward(const float *x, Tensor *gain,
                          const float *inverse_rms, const float *dy,
                          float *dx, int rows, int columns) {
    int row;
    for (row = 0; row < rows; ++row) {
        int col;
        float dot = 0.0f;
        const float r = inverse_rms[row];
        for (col = 0; col < columns; ++col) {
            const int index = row * columns + col;
            dot += dy[index] * gain->value[col] * x[index];
            gain->grad[col] += dy[index] * x[index] * r;
        }
        for (col = 0; col < columns; ++col) {
            const int index = row * columns + col;
            const float u = dy[index] * gain->value[col];
            dx[index] += r * u - x[index] * r * r * r * dot / (float)columns;
        }
    }
}
```

2 things we want to calculate

The **gradient** of **gain** to update **gain**

And also the dx to give to the last layer

$$\frac{\partial L}{\partial g_i} = dy_i \cdot x_i \cdot r$$

Gain:

$$dx_i = ru_i - x_i \frac{r^3}{N} \sum_j dy_j g_j x_j$$

dx:

dx 和 x 同形状 **gain.grad** 和 **gain.value** 同形状

32 个样本并行 forward → 32 个 loss → 平均成 1 个 loss → 一次 backward → 得到相当于 32 个样本梯度平均后的 **gradient** → 更新一次参数

4. Attention

First understand the structure

For **attention** layer the input is still [t,d]

And the output is also [t,d]

Q v k each has a size of [d,d]

So obv **wq wv wk** is also [t,d]

And then we do some operation to put **wq wv wk** tgt and we get the final matrix [t,d]

First we can focus on one layer for example **q**

So basically **q** means **query**: what im looking for?

When we do the operation **wq**, the result we get means: what im looking for

How to understand:

Q is a matrix which is trained to input a token and output what its looking for

设 A 的每一行是：

$$A = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_t \end{bmatrix}$$

那么：

$$AB = \begin{bmatrix} a_1 B \\ a_2 B \\ \vdots \\ a_t B \end{bmatrix}$$

So for each token(the hidden layer(vector)) becomes another vector which represents what its looking for

This works for all the **q v k**

So basically each vector means sth else

And then

$$QK^{\top} = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_t \end{bmatrix} \begin{bmatrix} k_1^{\top} & k_2^{\top} & \dots & k_t^{\top} \end{bmatrix}$$
$$S = \begin{bmatrix} q_1 \cdot k_1 & q_1 \cdot k_2 & \dots & q_1 \cdot k_t \\ q_2 \cdot k_1 & q_2 \cdot k_2 & \dots & q_2 \cdot k_t \\ \vdots & \vdots & \ddots & \vdots \\ q_t \cdot k_1 & q_t \cdot k_2 & \dots & q_t \cdot k_t \end{bmatrix}$$

Realize how we retake **K** so we can compare every token in **Q** with every token in **k** and get the **attention score** for every token to every token

Also for every token we only show the previous token and it cannot c the future tokens so the matrix would look like this

devide by d so we wont make the **attention score** too big

为什么维度越大，点积越大？

如果 $D = 2$ ，点积只加两项：

$$Q \cdot K = q_0 k_0 + q_1 k_1$$

如果 $D = 1000$ ，就要加 1000 项：

$$Q \cdot K = q_0 k_0 + q_1 k_1 + \cdots + q_{999} k_{999}$$

加的项越多，结果通常越容易出现较大的正数或负数。

例如：

D 较小时，分数可能是： $[1, 2, 3]$
D 较大时，分数可能是： $[10, 20, 30]$



分数太大有什么问题？

这些分数后面要进入 softmax。

对于：

$$[1, 2, 3]$$

softmax 大约是：

$$[0.09, 0.24, 0.67]$$

每个 token 都还保留一定注意力。

但如果分数放大十倍：

$$[10, 20, 30]$$

softmax 大约变成：

$$[0.000000002, 0.000045, 0.999955]$$

几乎变成只关注一个 token。

这会使 softmax 过早进入非常极端的状态，训练时梯度也会变得很小。

所以需要把分数缩小：

$$QK^T$$

$$\begin{bmatrix} S_{00} & \text{禁止} & \text{禁止} \\ S_{10} & S_{11} & \text{禁止} \\ S_{20} & S_{21} & S_{22} \end{bmatrix}$$

Ok now we have an attention matrix which looks like this

$$P = \begin{bmatrix} 1 & 0 & 0 \\ 0.25 & 0.75 & 0 \\ 0.10 & 0.30 & 0.60 \end{bmatrix}$$

We softmax() each row and get this sth like this:

Again this is the **attention score** for token to every token

And then we do $P \cdot V$

We also use this intuition

$$QK^T = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_t \end{bmatrix} \begin{bmatrix} k_1^T & k_2^T & \cdots & k_t^T \end{bmatrix}$$

$$S = \begin{bmatrix} q_1 \cdot k_1 & q_1 \cdot k_2 & \cdots & q_1 \cdot k_t \\ q_2 \cdot k_1 & q_2 \cdot k_2 & \cdots & q_2 \cdot k_t \\ \vdots & \vdots & \ddots & \vdots \\ q_t \cdot k_1 & q_t \cdot k_2 & \cdots & q_t \cdot k_t \end{bmatrix}$$

So $O_{i,c}$ 表示: token i 从所有可见 token 的 **Value** 第 c 个特征中, 加权收集到的结果。

This shape of O : [t d]

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d_{\text{head}}}} + \text{mask} \right) V$$

输出就是 o

```
int minillm_attention_backward(const float *q, const float *k, const float *v,
                             const float *probability, const float *d_output,
                             float *d_q, float *d_k, float *d_v,
                             int token_count, int d_model, int heads) {
    const int head_dim = d_model / heads;
    const float scale = 1.0f / sqrtf((float)head_dim);
    float *d_probability = minillm_calloc((size_t)token_count, sizeof(float));
    int i;
    if (d_probability == NULL) return 0;

    for (i = 0; i < token_count; ++i) {
        int head;
        for (head = 0; head < heads; ++head) {
            int j;
            float probability_dot = 0.0f;
            const size_t row_base = ((size_t)head * (size_t)token_count +
                                     (size_t)i * (size_t)token_count);

            /* output=P@V, 所以 dP=d_output@V, dV=P^T@d_output. */
            for (j = 0; j <= i; ++j) {
                int c;
                float dp = 0.0f;
                const float p = probability[row_base + (size_t)j];
                for (c = 0; c < head_dim; ++c) {
                    const int channel = head * head_dim + c;
                    dp += d_output[i * d_model + channel] *
                        v[j * d_model + channel];
                    d_v[j * d_model + channel] +=
                        p * d_output[i * d_model + channel];
                }
                d_probability[j] = dp;
                probability_dot += p * dp;
            }

            /* softmax 导数: dscore[j]=P[j]*(dP[j]-sum(P*dP)). */
            for (j = 0; j <= i; ++j) {
                int c;
                const float p = probability[row_base + (size_t)j];
                const float d_score = p * (d_probability[j] - probability_dot) * scale;

                /* score=Q.K/sqrt(Dh), 所以梯度分别乘另一个向量. */
                for (c = 0; c < head_dim; ++c) {
                    const int channel = head * head_dim + c;
                    d_q[i * d_model + channel] +=
                        d_score * k[j * d_model + channel];
                    d_k[j * d_model + channel] +=
                        d_score * q[i * d_model + channel];
                }
            }
        }
    }
    free(d_probability);
    return 1;
}
```

反向传播

从 $O=PV$ 求 dP 和 dV

通过 **softmax** 求 dS

3. 从 $S=QK^T/d_h$ 求 dQ, dK